

Hệ thống phân vùng thực thể chẩn đoán bệnh trên lá cây nông nghiệp đặc sản cà phê và lúa

CSC14005 - Nhập môn học máy

Sinh viên thực hiện:

Nguyễn Hồ Anh Tuấn - 23120185

Lê Xuân Trí - 23120099

Đàm Tiến Đạt - 23120118

Tổng Thanh Phúc - 23120158

Dương Tuấn Anh - 23120208

Giảng viên hướng dẫn:

Thầy Bùi Tiến Lên

Khoa Công nghệ Thông tin
Trường Đại học Khoa học Tự nhiên - ĐHQG-HCM

Mục lục

- ➊ Giới thiệu
- ➋ Thu thập và phân tích dữ liệu
- ➌ Lựa chọn và huấn luyện mô hình
- ➍ Kết quả và Thảo luận
- ➎ Xây dựng và Triển khai Ứng dụng

- ➊ Giới thiệu
- ➋ Thu thập và phân tích dữ liệu
- ➌ Lựa chọn và huấn luyện mô hình
- ➍ Kết quả và Thảo luận
- ➎ Xây dựng và Triển khai Ứng dụng

Bối cảnh nông nghiệp Việt Nam

Lúa gạo và cà phê Robusta là hai nông sản đặc sản xuất khẩu chủ lực của Việt Nam. Dịch bệnh hại lá thường xuyên đe dọa nghiêm trọng đến sinh kế của hàng triệu hộ nông dân tại Đồng bằng sông Cửu Long và Tây Nguyên.

Thách thức trong canh tác

Nông dân khó tiếp cận chuyên gia bệnh học, thường chẩn đoán cảm quan dẫn đến sử dụng sai hoặc lạm dụng thuốc bảo vệ thực vật. Điều này làm tăng chi phí và gây ô nhiễm môi trường nghiêm trọng.

Phân tích Vấn đề - Phát biểu bài toán

Bài toán được mô hình hóa dưới dạng phân vùng thực thể (*instance segmentation*) với:

- Đầu vào: Ảnh lá cây nông nghiệp $I \in \mathbb{R}^{H \times W \times 3}$
- Đầu ra: Tập hợp K thực thể bệnh $E = \{e_1, e_2, \dots, e_K\}$

Trong đó, mỗi thực thể vết bệnh $e_k = (b_k, c_k, m_k)$ bao gồm hộp bao b_k , nhãn c_k và mặt nạ m_k .

Thách thức trong điều kiện thực tế bao gồm vết bệnh nhỏ trên nền phức tạp, độ tương đồng thị giác cao ở giai đoạn sớm, và điều kiện ánh sáng tự nhiên biến động mạnh.

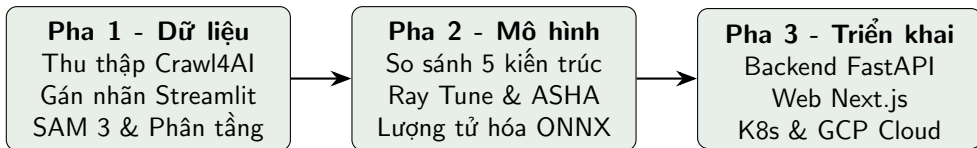
Định lượng & Học máy

Xây dựng bộ dữ liệu 7.149 ảnh thuộc 8 lớp bệnh. Huấn luyện so sánh hiệu năng 5 kiến trúc phân vùng thực thể, và tinh chỉnh siêu tham số (Ray Tune & ASHA) nhằm đạt độ chính xác tối ưu.

Thực tiễn & Triển khai

Phát triển ứng dụng Web chẩn đoán thời gian thực với độ trễ dưới 200 ms/ảnh. Triển khai cụm Kubernetes (k3s) trên GCP có chứng chỉ bảo mật SSL và tự động hóa CI/CD.

Tổng quan về Phương pháp



Hình 1: Quy trình phát triển hệ thống end-to-end

Hệ thống được thiết kế theo quy trình end-to-end khép kín, đảm bảo tính nhất quán từ dữ liệu thô được thu thập trong môi trường thực tế cho tới ứng dụng chạy thực tế trên hạ tầng đám mây.

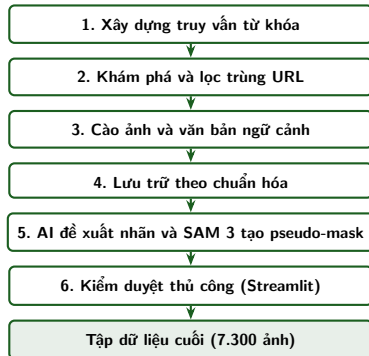
Mục lục

- ① Giới thiệu
- ② Thu thập và phân tích dữ liệu
- ③ Lựa chọn và huấn luyện mô hình
- ④ Kết quả và Thảo luận
- ⑤ Xây dựng và Triển khai Ứng dụng

2.1. Nguồn và phương pháp thu thập

Nguồn và mở rộng dữ liệu

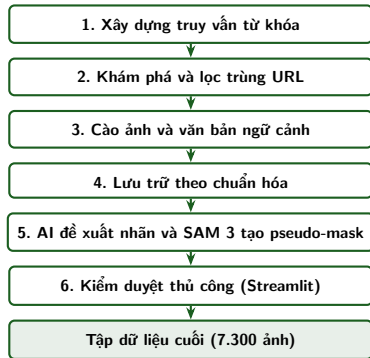
- Kế thừa từ hai bộ tập dữ liệu có sẵn: Rice Diseases Image Dataset và Detect Disease On Coffee Leaves.
- Thu thập bổ sung qua hình thức cào dữ liệu từ các trang web, blog chuyên ngành bằng công cụ Crawl4AI.
- Khám phá và lọc URL tự động nhờ DuckDuckGo và Serper.dev.



2.2. Nguồn và phương pháp thu thập

Vòng lặp có sự tham gia của con người

- Gán nhãn theo hướng vòng lặp có sự tham gia của con người.
- AI đề xuất nhãn (Gemma 4) và tạo pseudo-mask tự động bằng mô hình SAM 3.
- Kiểm duyệt thủ công trên công cụ Streamlit để xác nhận hoặc sửa nhãn.



2.2. Nguồn và phương pháp thu thập

Quy mô bộ dữ liệu hợp nhất

Tập dữ liệu thu thập được bao gồm 7.300 ảnh với tổng cộng 7.562 COCO format annotation phục vụ cho bài toán instance segmentation.

Bảng 1: Quy mô bộ dữ liệu hợp nhất

Miền cây trồng	Số lượng ảnh	Số annotation
Lúa	3.421	3.720
Cà phê	3.879	3.842
Tổng cộng	7.300	7.562

2.3. Tiền xử lý và làm sạch

Quy trình kiểm duyệt hai bước

Dữ liệu thô thu thập từ nhiều nguồn được lọc qua quy trình kiểm duyệt chất lượng nghiêm ngặt:

- 1 Khử trùng lặp: tính toán hàm băm MD5 trên nội dung nhị phân để phát hiện và loại bỏ chính xác các ảnh trùng lặp hoàn toàn.
- 2 Lọc ảnh mờ nhòe: đánh giá độ sắc nét dựa trên phương sai của phép biến đổi Laplacian với ngưỡng loại bỏ $\tau = 100$.

Bảng 2: Kết quả làm sạch toàn bộ tập ảnh

Lý do xử lý	Số ảnh	Tỷ lệ (%)
Ảnh trùng lặp	1	0,01
Ảnh mờ nhòe	150	2,05
Giữ lại sử dụng	7.149	97,93

2.3. Tiền xử lý và làm sạch

Phân bố chi tiết số ảnh sau khi làm sạch

Bảng 3: Phân bố số lượng ảnh và tỷ lệ theo lớp và miền sau làm sạch

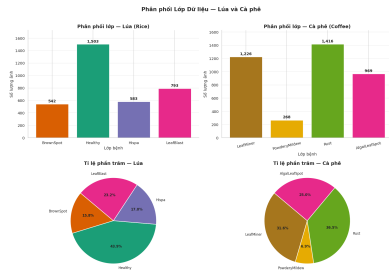
Miền dữ liệu	Lớp bệnh	Số lượng ảnh	Tỷ lệ trong miền (%)
Lúa (3.351 ảnh)	Lá khỏe mạnh	1.472	43,93
	Bệnh đạo ôn	777	23,19
	Sâu đục lá	571	17,04
	Bệnh đốm nâu	531	15,85
Cà phê (3.798 ảnh)	Bệnh rỉ sắt	1.387	36,52
	Sâu đục lá	1.200	31,60
	Bệnh đốm rong	949	24,99
	Bệnh phấn trắng	262	6,90

2.4. Phân tích và khám phá dữ liệu

Mất cân bằng lớp và hình học ảnh

- Mất cân bằng lớp: tỷ số mất cân bằng $IR = N_{\max}/N_{\min}$ trên Lúa là 2,77 và trên Cà phê là 5,28 (lớp phân trắng chỉ chiếm 6,9%).
- Chênh lệch độ phân giải: ảnh lúa trung bình 2.031×2.026 px, chủ yếu vuông; ảnh cà phê nhỏ hơn rõ rệt, 971×1.154 px, thiên về khung dọc.

⇒ Chuẩn hóa kích thước ảnh giữ nguyên tỷ lệ khung hình thực bằng kỹ thuật letterbox. Sử dụng các kỹ thuật tăng cường dữ liệu.



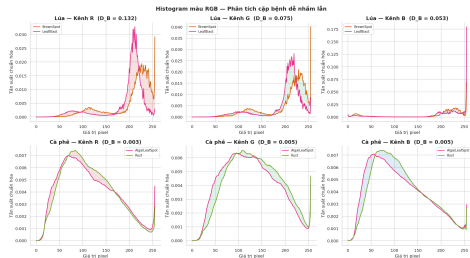
Hình 2: Phân bố số lượng và tỷ lệ các lớp bệnh

2.4. Phân tích và khám phá dữ liệu

Màu sắc và cặp bệnh dễ nhầm lẫn

- Đặc trưng chiếu sáng: ảnh Lúa sáng rõ rệt (độ sáng trung bình 198 - 202) so với ảnh Cà phê (116 - 129). Kênh Green chiếm ưu thế nhưng độ tương phản giữa các lớp giao thoa mạnh.
- Khoảng cách Bhattacharyya D_B trên histogram RGB chuẩn hóa:
 - Lúa: cặp đốm nâu và đạo ôn có D_B nhỏ trên cả 3 kênh (lớn nhất ở kênh Red với $D_B = 0,132$).
 - Cà phê: cặp đốm rong và rỉ sắt có phân bố màu gần trùng khớp hoàn toàn ($D_B \approx 0,003 - 0,005$).

⇒ Mô hình không thể dựa vào màu toàn cục mà bắt buộc phải học kết cấu cục bộ cùng ngữ cảnh không gian.



Hình 3: Histogram RGB của các cặp bệnh dễ nhầm lẫn

- ① Giới thiệu
- ② Thu thập và phân tích dữ liệu
- ③ Lựa chọn và huấn luyện mô hình**
- ④ Kết quả và Thảo luận
- ⑤ Xây dựng và Triển khai Ứng dụng

3.1. Chuẩn bị dữ liệu cho mô hình

Phân chia tập dữ liệu chống rò rỉ

Tập dữ liệu sạch gồm 7.149 ảnh được phân chia thành ba tập theo tỷ lệ chuẩn 70 : 15 : 15 bằng kỹ thuật Stratified Sampling.

Bảng 4: Phân bố số lượng ảnh theo tập con, miền và lớp bệnh

Miền dữ liệu	Train (70%)	Val (15%)	Test (15%)	Tổng cộng
Lúa	2.346	502	503	3.351
Cà phê	2.658	570	570	3.798
Tổng toàn cục (8 lớp)	5.004	1.072	1.073	7.149

3.1. Chuẩn bị dữ liệu cho mô hình

Tiền xử lý và tăng cường dữ liệu

Để tăng khả năng tổng quát hóa và chống overfitting, nhóm áp dụng các kỹ thuật chuẩn hóa và tăng cường dữ liệu trong quá trình huấn luyện:

Bảng 5: Các kỹ thuật tiền xử lý và tăng cường dữ liệu

Phương pháp	Chi tiết thiết lập và tác dụng
Letterbox	Đưa ảnh về kích thước 640×640 hoặc 256×256 px giữ nguyên tỷ lệ khung hình thực, tránh méo dạng vùng bệnh.
Cắt và đổi kích thước Lật hình học	Cắt và thay đổi kích thước ngẫu nhiên với tỷ lệ diện tích từ 0,75 đến 1,0. Lật ngang với xác suất $p = 0,5$, lật dọc với xác suất $p = 0,2$.
Biến đổi affine	Xoay ngẫu nhiên trong khoảng góc $\pm 30^\circ$.
Biến thiên quang học	Điều chỉnh ngẫu nhiên độ sáng và màu sắc ảnh.
Trộn mẫu (<i>MixUp</i>)	Mixup với hệ số $\alpha = 0,4$.

3.2. Lựa chọn và kiến trúc mô hình

Nhóm khảo sát, huấn luyện và so sánh năm kiến trúc tiên tiến thuộc hai trường phái Convolutional Neural Network (CNN) và Vision Transformer (ViT) nhằm tối ưu điểm cân bằng giữa độ chính xác và tốc độ suy luận:

Bảng 6: Tổng quan năm kiến trúc cho bài toán instance segmentation

Mô hình	Họ	Đặc điểm nổi bật và lý do lựa chọn
Mask R-CNN [?]	CNN	Kiến trúc hai giai đoạn kinh điển, mang lại độ chính xác cao, được chọn làm baseline đối chiếu hiệu suất.
YOLO26m-seg [?, ?]	CNN	Kiến trúc một giai đoạn với tốc độ thời gian thực xuất sắc, cỡ medium tối ưu điểm cân bằng chính xác - tốc độ.
RF-DETR [?, ?]	ViT	Thiết kế end-to-end loại bỏ NMS, DINOv2 feature extraction, real-time Transformer.
MobileSAM [?, ?]	ViT	Phiên bản thu gọn siêu nhẹ của SAM, khả năng zero-shot với câu lệnh đầu vào, tối ưu cho edge devices.
Mask2Former [?]	ViT	Kiến trúc phân vùng hợp nhất dựa trên mask classification, đạt hiệu năng đa tác vụ vượt trội.

3.3. Cấu hình huấn luyện

Thiết lập cấu hình chung

Bảng 7: Cấu hình huấn luyện chung

Tham số	Giá trị thiết lập
Phần cứng GPU	NVIDIA Tesla T4
Tỷ lệ chia dữ liệu	70 : 15 : 15
Thuật toán tối ưu	AdamW
Effective batch size	8
Số epoch tối đa	30
Dừng sớm	5
Seed	3407

Tham số riêng theo kiến trúc

Bảng 8: Cấu hình riêng cho từng mô hình

Mô hình	lr_0	Kích thước	Ghi chú kỹ thuật
Mask R-CNN	$1 \cdot 10^{-4}$	256×256	Batch 2, grad acc 4
YOLO26m-seg	$8 \cdot 10^{-4}$	640×640	Batch 8
RF-DETR	$1 \cdot 10^{-4}$	640×640	Batch 2, grad acc 4
MobileSAM	$1 \cdot 10^{-5}$	1024×1024	Batch 2, grad acc 4
Mask2Former	$5 \cdot 10^{-5}$	384×384	Batch 4, grad acc 2

*Ghi chú: Batch: batch size, grad acc: tích lũy đạo hàm.

3.4. Tinh chỉnh siêu tham số

Tối ưu tự động bằng Ray Tune

- Tinh chỉnh siêu tham số cho mô hình tốt nhất YOLO26m-seg bằng Ray Tune.
- Áp dụng thuật toán lập lịch bất đồng bộ ASHA.
- Đánh giá song song nhiều cấu hình thử nghiệm, dừng sớm các cấu hình kém hiệu quả ngay từ những epoch đầu giúp tối ưu hóa tài nguyên tính toán.
- Thực nghiệm 20 thử nghiệm độc lập trên không gian sáu siêu tham số cốt lõi.

Bảng 9: Không gian tìm kiếm siêu tham số

Siêu tham số	Phân phối	Khoảng/Tập giá trị
Learning rate (lr_0)	Log-uniform	$[1 \cdot 10^{-5}, 1 \cdot 10^{-2}]$
Momentum	Uniform	$[0, 85, 0, 95]$
Weight decay	Log-uniform	$[1 \cdot 10^{-5}, 1 \cdot 10^{-3}]$
Warmup epochs	Choice	$\{1, 2, 3\}$
Hệ số khung bao λ_{box}	Uniform	$[5, 0, 10, 0]$
Hệ số phân loại λ_{cls}	Uniform	$[0, 3, 0, 8]$

Mục lục

- ① Giới thiệu
- ② Thu thập và phân tích dữ liệu
- ③ Lựa chọn và huấn luyện mô hình
- ④ Kết quả và Thảo luận**
- ⑤ Xây dựng và Triển khai Ứng dụng

So sánh hiệu năng 5 mô hình

Bảng 10: Tóm tắt kết quả mAP@50 và Thời gian suy luận

Mô hình	Tập Lúa (Rice)		Tập Cà phê (Coffee)	
	mAP@50	Thời gian (ms)	mAP@50	Thời gian (ms)
MobileSAM	0.5637	1487.54	0.6923	1616.84
Mask R-CNN	0.5966	760.50	0.9480	702.36
Mask2Former	0.6971	96.81	0.9200	62.88
RF-DETR	0.7038	198.31	0.8428	70.31
YOLO26-seg	0.8247	65.72	0.9012	20.21

- YOLO26-seg đạt độ chính xác mAP đứng hạng đầu trên tập Lúa và hạng ba trên tập Cà phê.
- YOLO26-seg sở hữu tốc độ suy luận vượt trội nhất (20 - 65 ms/ảnh) và kích thước gọn nhẹ nhất (6.23 MB).

Kết quả tinh chỉnh YOLO26-seg bằng RayTune và ASHA

Bảng 11: So sánh trước và sau tinh chỉnh siêu tham số

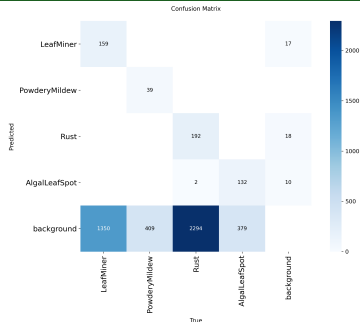
Tập dữ liệu	Trạng thái	mAP@50	mAP@50:95	mIoU	Dice	Inference (ms)	Size (MB)
Lúa (Rice)	Baseline	0.8247	0.7891	0.8543	0.8712	65.72	6.23
	Tuned	0.8481	0.8034	0.8697	0.8856	65.41	6.23
Cà phê (Coffee)	Baseline	0.9012	0.8637	0.8921	0.9087	20.21	6.23
	Tuned	0.9234	0.8845	0.9053	0.9192	19.85	6.23

- Phương pháp tìm kiếm kết hợp thuật toán loại bỏ sớm ASHA giúp tăng hơn 2% mAP@50 trên cả hai miền dữ liệu.

Ma trận nhầm lẫn (Confusion Matrix) của YOLO26-seg



Tập Lúa



Tập Cà phê

Hình 4: Ma trận nhầm lẫn chuẩn hóa của mô hình YOLO26-seg

- Nhầm lẫn điển hình: Đốm nâu (BrownSpot) ↔ Bọ xít gai (Hispa) trên lúa.
- Cà phê: Nhầm lẫn nhẹ giữa Rỉ sắt (Rust) ↔ Đốm rong (AlgalLeafSpot).

Phân tích chuyên sâu (Overfitting / Underfitting)

Hiện tượng lệch khớp

- Hiện tượng chưa khớp (underfitting) xảy ra với MobileSAM do mô hình zero-shot khó nhận diện các vết bệnh nhỏ trên lá lúa.
- Hiện tượng khớp quá mức (overfitting) xảy ra ở Mask R-CNN và Mask2Former do kích thước mô hình quá lớn so với tập dữ liệu.

Giải pháp khắc phục

Để nâng cao hiệu năng và tính tổng quát hóa:

- Tăng cường tập dữ liệu huấn luyện (Data Augmentation) hoặc bổ sung thêm mẫu thực tế.
- Áp dụng các kỹ thuật Regularization mạnh mẽ hơn như Dropout, Weight decay và Early Stopping.

Mục lục

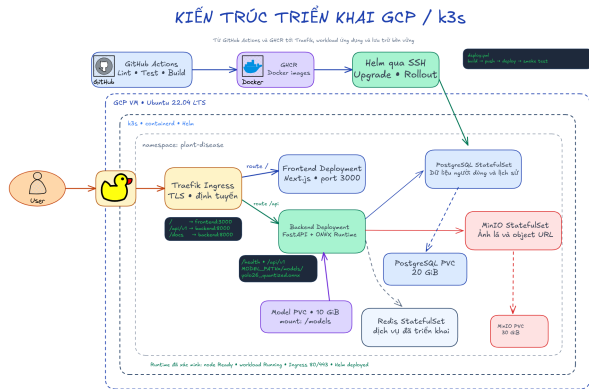
- ① Giới thiệu
- ② Thu thập và phân tích dữ liệu
- ③ Lựa chọn và huấn luyện mô hình
- ④ Kết quả và Thảo luận
- ⑤ Xây dựng và Triển khai Ứng dụng

Kiến trúc hệ thống và tích hợp mô hình

Mô hình tổng thể

Tích hợp mô hình YOLO26-seg đã lượng tử hóa dạng ONNX để thực thi suy luận trên CPU.

Hệ thống chia làm ba thành phần độc lập: giao diện Next.js, backend FastAPI điều phối ONNX Runtime, lưu trữ với PostgreSQL và MinIO.

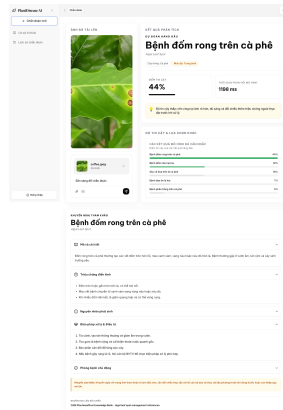


Hình 5: Kiến trúc tổng thể hệ thống trên GCP và k3s

Các tính năng chính

Hệ thống hỗ trợ xác thực tài khoản qua JWT thời hạn 24 giờ.

Nông dân tải ảnh hoặc chụp trực tiếp từ camera. Ảnh được gửi đến API để nhận diện mặt nạ bệnh kèm độ tin cậy. Kết quả được lưu lịch sử và hiển thị khuyến nghị từ chuyên gia.



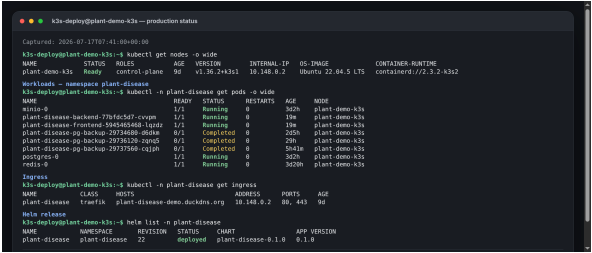
Hình 6: Giao diện kết quả chẩn đoán bệnh lá cà phê

Triển khai hạ tầng đám mây và k3s

Hạ tầng Kubernetes

Workload được container hóa toàn bộ và quản lý bởi k3s trên máy ảo GCP.

Cấu hình cụm dịch vụ qua Helm Chart, định tuyến Ingress Traefik và tự động cấp chứng chỉ SSL HTTPS Let's Encrypt qua cert-manager để bảo mật toàn diện.



```
k3s-deploy@plant-demo-k3s:~$ kubectl get nodes -o wide
NAME                STATUS    ROLES    AGE   VERSION   INTERNAL-IP   OS-IMAGE               CONTAINER-RUNTIME
plant-demo-k3s      Ready    control-plane   9d    v1.36.2+k3s1  10.148.0.2    Ubuntu 22.04.5 LTS     containerd://2.3.2-k3s2

Workloads - namespace plant-disease
k3s-deploy@plant-demo-k3s:~$ kubectl -n plant-disease get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   NODE
minio-0                             1/1    Running   0           3d2h  plant-demo-k3s
plant-disease-backend-77bfc5d7-cvypm 1/1    Running   0           19m   plant-demo-k3s
plant-disease-frontend-594346346b-lqpdz 1/1    Running   0           19m   plant-demo-k3s
plant-disease-pg-backup-29734688-dsdkm 0/1    Completed 0           2d5h  plant-demo-k3s
plant-disease-pg-backup-29736128-znqg5 0/1    Completed 0           29h   plant-demo-k3s
plant-disease-pg-backup-29737568-cqjsh 0/1    Completed 0           5h41m plant-demo-k3s
postgres-0                          1/1    Running   0           3d2h  plant-demo-k3s
redis-0                              1/1    Running   0           3d20h plant-demo-k3s

Ingress
k3s-deploy@plant-demo-k3s:~$ kubectl -n plant-disease get ingress
NAME          CLASS    HOSTS                                ADDRESS          PORTS   AGE
plant-disease traefik  plant-disease-demo.duckdns.org      10.148.0.2      80, 443  9d

Helm release
k3s-deploy@plant-demo-k3s:~$ helm list -n plant-disease
NAME          NAMESPACE   REVISION   STATUS   CHART               APP VERSION
plant-disease plant-disease 22         deployed plant-disease-0.1.0 0.1.0
```

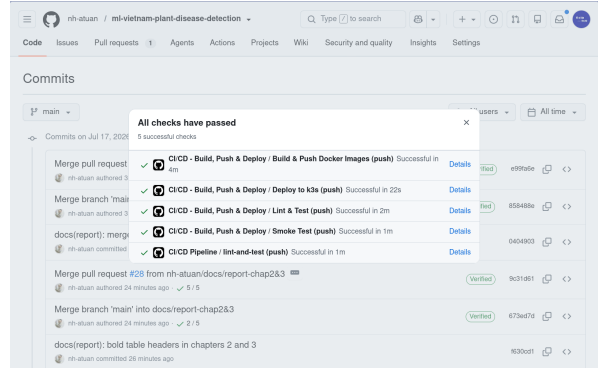
Hình 7: Trạng thái vận hành của các cụm pod trên k3s

Quy trình tích hợp và triển khai tự động

Tự động hóa CI/CD

Xây dựng workflow CI/CD hoàn chỉnh trên GitHub Actions, kiểm tra định dạng và chạy kiểm thử mã nguồn.

Đóng gói và đẩy Docker image lên GHCR, kết nối SSH cập nhật Helm Chart tại VM và kiểm tra khép kín qua Smoke Test.



Hình 8: Quy trình CI/CD hoàn tất thành công

Mục lục

- ① Giới thiệu
- ② Thu thập và phân tích dữ liệu
- ③ Lựa chọn và huấn luyện mô hình
- ④ Kết quả và Thảo luận
- ⑤ Xây dựng và Triển khai Ứng dụng

6.1. Tóm tắt Kết quả

6.2. Hạn chế

6.3. Hướng phát triển

Tài liệu tham khảo I



K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask R-CNN," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 2961–2969, 2017.



G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLO," 2023.



G. Jocher and A. Chaurasia, "YOLO real-time instance segmentation architecture and best practices," *Ultralytics Technical Reports*, 2024.



N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *European Conference on Computer Vision (ECCV)*, pp. 213–229, Springer, 2020.



W. Lv, Y. Zhao, S. Xu, J. Wei, G. Wang, C. Cui, Y. Du, Q. Dang, and S. Liu, "DETRs beat YOLOs on real-time object detection," *arXiv preprint arXiv:2304.08069*, 2023.



C. Zhang, D. Han, Y. Qiao, J. H. Kim, S.-H. Bae, S. Lee, and C. S. Hong, "Faster segment anything: Towards lightweight SAM for mobile applications," *arXiv preprint arXiv:2306.14289*, 2023.



A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. Dollár, and R. Girshick, "Segment anything," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 4015–4026, 2023.



B. Cheng, I. Misra, A. G. Schwing, A. Kirillov, and R. Girdhar, "Masked-attention mask transformer for universal image segmentation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1290–1299, 2022.